

AI SOP 协作流程设计

核心判断：AI 写出来的代码本身质量不是关键问题，关键问题是**代码生产速度超过了规范传递速度**。这套框架的作用是让规范能跟上 AI 的速度——通过文件而不是口头约定来传递，并且文档本身是有反馈迭代机制的，是 AI SOP 的记忆系统的一部分：每次交付产出都会写回框架，框架因此持续收敛而不是漂移。

框架构成：两个 kit，一个边界

框架由两个独立 kit 组成，零耦合，可单独使用，也可组合。

Kit	回答的问题	核心内容
arch-kit	仓库长什么样	目录结构、契约分层、服务脚手架、文档模板
ai-kit	AI 怎么参与工作	Skills、配置模板、交付 SOP、Playbook

两个 kit 刻意分开的原因：**骨架与 AI 工作流的升级节奏不同**。仓库结构和契约分层一旦稳定，几乎不会频繁变动；但 AI 工具每个季度都在换。捆绑在一起，每次换 vendor 就要碰仓库骨架，代价太高。

arch-kit 和 ai-kit 之间只有一个约定：arch-kit 的 `AGENTS.md` 里写清楚 AI 需要知道的仓库规则，ai-kit 的 Skills 读它——除此之外没有任何依赖。

核心设计原则

上下文是稀缺资源，不是越多越好

AI agent 工作的质量不只取决于模型能力，更取决于它在当前对话里拿到什么信息。但 context window 是有成本的：放进去的内容越多，模型对其中每一条信息的精确召回越差。"把所有文档都丢给 AI"不是正确做法。

框架的应对是：**按生命周期分层投喂**。

System 层	组织级红线、安全策略	← 全局生效，不随任务变动
Project 层	AGENTS.md、规范、Skills	← 仓库级，团队共享
Session 层	当次任务的 Plan、笔记	← 任务结束即归档，不污染下次

对应地，仓库文档同样三层：L1 是单次需求草稿，L2 是服务级长期文档，L3 是全局 SSOT（契约 + 标准 + 架构）。**L1 完成后必须回写 L2/L3**——这是知识从"一次性用完"变成"团队永久资产"的唯一机制。

长任务还有三种专门的上下文对策：

策略	作用	在框架里的对应
Compaction	压缩历史，保留决策依据	ADR 让下一个 AI 继承"为什么"
Structured Note-Taking	结构化记录，防止遗忘	<code>retro-audit</code> skill 在 T7 沉淀关键决策
Subagent Fan-out	并行探索，主 agent 只看摘要	<code>bc-impact-map</code> skill 拆分边界上下文分析

文件角色显式声明

每个文档首行强制标注角色徽章，目的是让 AI 和人都清楚"这个文件能做什么"：

SSOT — 唯一权威来源，所有引用指向这里
 VIEW — 叙述性内容，禁止复制 SSOT 的 schema 或枚举
 SPEC — 单次草稿，完成后回写到目标并归档
 AUTO — 机器生成，禁止手动编辑

没有这套标注，AI 会把 VIEW 文件里的内容当权威来源，把 AUTO 文件当成可以手动调整的配置——这两种误操作都会悄悄引入不一致，而且难以排查。

Harness 优先于 Model

模型能力会持续提升，但每次提升都会带来行为变化。如果团队的"AI 使用方式"建立在对某个特定模型行为的依赖上，每次升级就是一次重新适应。

正确做法是把可重用的部分封装在 **Harness** 里：Skills 封装 workflow 步骤（不是 prompt 字符串），AGENTS.md 封装团队约定，Hooks 封装质量红线。模型换了，这些全部保留，只需要校准少量行为差异。

Skills 的触发条件、执行步骤、输出格式都在 SKILL.md 里定义，不依赖任何特定模型版本的行为。

跨 vendor 分三层，不做统一翻译器

四家主流 agent（Claude Code / Cursor / Copilot / Codex）配置格式各异，不存在一份配置自动同步四家的路径。强行统一会制造隐性维护负担——任何一家格式变动都要改"统一层"。

正确策略是按通用性分层维护：

层	内容	做法
真通用	仓库入口规则（AGENTS.md）	一份文件，四家全读
准通用	工作流（Skills）	一份 SKILL.md，symlink 到四家路径
各家原生	Hooks / MCP / Settings	各自目录独立，不互相派生

SOP 与 Playbook 分层，升级互不干扰

- **SOP（T0–T7）**：回答"做什么 / 谁负责 / Gate 在哪"。删掉所有 AI 工具，SOP 仍是一份完整可执行的人工流程。
- **Playbook**：回答"用哪个 agent / 哪个 Skill / 具体操作步骤"。AI 工具换代，只改 Playbook，SOP 不动。

这个分层的价值：团队协作流程的稳定性不受 AI 工具迭代速度影响。

当前状态与落地路径

当前状态：设计与文档已完成，`install.sh` 在 `im-mesh-platform` 参考实现中跑通了端到端安装流程（10 步、四家 vendor 目录对等、spec-kit 集成）。完整的 T0–T7 周期尚未在真实团队、真实需求下运行过。

尚待验证的环节：

环节	还不清楚什么
L1 → L2/L3 回写纪律	多人并发时是否真的会执行，还是悄悄跳过
三层门禁的 CI 配置	覆盖率阈值和 SAST 规则需要结合具体技术栈校准
AGENTS.md 作为单一共识源	团队成员日常使用时会不会绕过它，或者各自另起配置

落地顺序（待做）：

- ☐ 用一个简单的真实项目完整跑一遍 T0–T7，记录卡点
- ☐ T1 影响地图和 T2 契约先行是最高价值的两步，优先做扎实
- ☐ T7 的 `retro-audit` skill 必须执行——每次复盘产出直接修正框架，这是唯一的自我完善机制
- ☐ 根据实操发现补齐 `gate-checklist` 内容、`constitution.md.tmpl` 模板

框架的每个步骤可以单独采用，不强制一次性全部落地。

方向验证（进行中）

设计逻辑自洽，但还需要外部印证。目标是找到**聪明人做了不同选择**的案例，不是找支持现有结论的材料。

- ☐ 找已有 AGENTS.md + Skills/Hooks + 结构化 SOP 的工程团队仓库，对比上下文分层方式和 Harness 资产选择
- ☐ 找同类 SOP 驱动 AI 工程框架，重点关注与本框架的分歧点
- ☐ 找 AI workflow 设计的深度文章，核查 T0–T7 步骤粒度和 context 三层划分是否有更成熟的对标
- ☐ 专门收集失败/弃用案例——流程太重、团队不执行、工具本身——评估本框架是否会踩同样的坑